

LEGALSAHAYAK

Capstone-II project report submitted
by the student of
Hybrid UG program in Computer Science & Data
Analytics

Student Name: Rishi Sharma
Roll No.: 2312res521
Group No.: 131

INDIAN INSTITUTE OF TECHNOLOGY PATNA
BIHTA - 801106, INDIA

Date: 30/03/2026

Declaration

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Date : 30/03/2026

Student Name: Rishi Sharma

Roll No.: 2312res521

Group No.: 131

Signature:rishisharma

Summary of the Project

The LegalSahyak project aims to build a **fully local legal question-answering system** that can process user queries and generate accurate responses using a hybrid retrieval and language model pipeline. The system is designed to work without dependency on external APIs, ensuring privacy, security, and efficiency.

The system takes legal queries as input and processes them through a multi-layer pipeline consisting of data retrieval, reranking, and answer generation. It uses a combination of **dense retrieval (FAISS)** and **sparse retrieval (BM25)** to fetch relevant legal documents such as statutes and contract clauses.

To improve the relevance of retrieved results, a **cross-encoder reranker** is used, which filters and ranks the most appropriate documents. These documents are then passed to a **local GGUF-based Large Language Model**, which generates the final response.

The backend is developed using **Node.js and Express**, and integrates Python-based machine learning components using the **child_process module**, enabling seamless execution of AI pipelines and at the uses a electron for creating “ .exe “ app for offline

Work done by each member:

1. Rishi Sharma (Backend Development & System Integration)

Rishi Sharma played a central role in the development of the LegalSahyak system, contributing to backend architecture, API design, and integration of AI components.

- Designed and implemented the complete backend system using Node.js and Express
- Developed REST APIs and routing for handling legal queries
- Integrated Python-based AI pipeline using child_process
- Managed execution of machine learning models from backend

- Designed end-to-end query processing workflow
 - Performed testing and debugging using Postman
 - Ensured smooth communication between backend and AI components
-

2. Pushkar Sharma

- Data scraping from legal sources
 - Data preprocessing and cleaning
 - Fine-tuning of Llama model
 - RAG pipeline development
-

3. Udit Sharma

- No significant contribution during mid-sem

4. Adnang Shashwat

- No significant contribution during mid-sem

5. Pragya Sharma

- No significant contribution during mid-sem

1.Introduction

The increasing complexity of legal information and the growing need for quick access to legal knowledge have led to the development of AI-based legal assistants. Traditional systems rely heavily on manual research or cloud-based APIs, which introduce challenges such as latency, dependency, and privacy concerns.

LegalSahyak is designed as a **local legal assistant system** that provides accurate responses without relying on external services. The system integrates modern AI techniques such as hybrid retrieval and large language models to deliver efficient and context-aware legal answers.

The project focuses on building a scalable architecture that combines backend engineering with machine learning integration.

Project Background and Aim

The rapid digitization of legal documents and the increasing demand for accessible legal knowledge have created a strong need for intelligent systems capable of understanding and retrieving legal information efficiently. Traditionally, legal research has been a time-consuming process requiring expertise in navigating statutes, acts, and contractual documents. With the advent of artificial intelligence, several systems have been developed to automate this process; however, most of them rely heavily on cloud-based APIs, which introduce limitations such as latency, dependency on internet connectivity, and concerns related to data privacy.

The LegalSahyak project is designed to address these challenges by building a fully local legal question-answering system that operates independently of external services. The aim of this project is to develop a system that can process user queries, retrieve relevant legal information using advanced retrieval techniques, and generate context-aware responses using a local language model. By integrating

backend engineering with machine learning pipelines, the project focuses on creating a scalable and efficient architecture that ensures both performance and security. Additionally, the system aims to bridge the gap between complex legal data and user-friendly query interfaces, making legal information more accessible to a wider audience.

Problem Statement

Despite significant advancements in artificial intelligence, existing legal question-answering systems face several critical limitations that hinder their effectiveness in real-world scenarios. Many systems depend on cloud-based infrastructures, which not only introduce latency but also raise serious concerns regarding the confidentiality of sensitive legal data. Furthermore, traditional retrieval systems often rely solely on keyword-based approaches, which fail to capture the contextual meaning of user queries, leading to inaccurate or irrelevant results.

Another major challenge lies in the integration of backend systems with machine learning pipelines. In many cases, these components operate independently, resulting in inefficient communication and increased system complexity. Additionally, most systems lack offline capabilities, making them unsuitable for environments where internet access is limited or restricted. The problem, therefore, is to design and implement a system that overcomes these limitations by combining efficient retrieval mechanisms, local model inference, and seamless backend integration, while maintaining high accuracy and scalability.

Objectives

The primary objective of the LegalSahyak project is to develop a robust and efficient legal question-answering system that can operate entirely in a local environment. The system aims to provide accurate and context-aware responses by leveraging hybrid retrieval techniques that combine both semantic and keyword-based search methods. Another key objective is to integrate a locally hosted large language model for answer generation, thereby eliminating dependency on external APIs and ensuring data privacy.

In addition to accuracy and privacy, the project also focuses on scalability and modularity, allowing future enhancements such as

frontend integration, dataset expansion, and performance optimization. The system is designed to handle real-world legal queries efficiently, ensuring that users receive relevant and reliable information in a timely manner. Furthermore, the project aims to demonstrate the practical implementation of integrating backend systems with machine learning pipelines, highlighting the importance of seamless communication between different components.

Solution Approach

The solution approach adopted in the LegalSahyak project is based on a multi-layered architecture that combines advanced retrieval techniques with local language model inference. The system begins by processing user queries through a backend API developed using Node.js and Express, which acts as the central component responsible for managing data flow and communication between different modules.

Once a query is received, it is passed to the retrieval layer, where a hybrid approach is used to fetch relevant legal documents. Dense retrieval using FAISS enables the system to capture the semantic meaning of queries, while sparse retrieval using BM25 ensures accurate keyword matching. The results obtained from these retrieval methods are then passed to a reranking layer, where a cross-encoder model evaluates and ranks the documents based on their relevance to the query.

After reranking, the most relevant documents are forwarded to the AI inference layer, where a local GGUF-based large language model generates the final response. This entire pipeline is executed through Python scripts, which are integrated with the Node.js backend using the `child_process` module. This integration allows seamless execution of the machine learning pipeline within the backend environment, ensuring efficient processing and response generation. The overall architecture is designed to be modular and scalable, allowing future enhancements and optimizations without significant changes to the existing system.

1. Data Collection and Processing

- The LegalSahyak system relies on structured legal datasets, including statutes, legal acts, and contract clauses collected from reliable public sources such as India Code and similar repositories.
- Raw legal documents are processed and converted into structured formats such as JSON to ensure efficient storage and retrieval.
- Documents are divided into meaningful sections such as clauses, headings, and subsections to improve precision in retrieval.
- Metadata is generated for each document, including information such as source, section title, and contextual identifiers to enhance search relevance.
- Data preprocessing involves normalization of text by removing encoding inconsistencies, special characters, and formatting issues.
- Python libraries such as pandas are used for large-scale data manipulation, cleaning, and structuring.
- The dataset is designed to support hybrid retrieval systems combining both semantic and keyword-based approaches.
- Separate indexing is maintained for different types of documents such as statutes and contracts to improve domain-specific retrieval.
- A data ingestion pipeline is developed to allow easy integration of new legal documents into the system.
- The overall goal of data processing is to provide a clean, structured, and retrieval-ready dataset for accurate query handling.

2. Input Layer (Layer 1)

- The input layer acts as the entry point of the system, where users submit legal queries in natural language format.
 - Backend APIs developed using Node.js and Express handle incoming user queries efficiently.
 - Input validation mechanisms ensure that invalid, empty, or malformed queries are not processed further.
 - Queries are standardized and formatted to ensure compatibility with downstream processing stages.
 - The system is designed to support flexible input formats for future scalability.
 - Asynchronous request handling is implemented to efficiently process multiple queries simultaneously.
 - Error handling ensures proper feedback is provided to users in case of invalid inputs.
 - The input layer serves as a bridge between user interaction and backend processing.
 - Query logging mechanisms are implemented to track inputs for debugging and system improvement.
 - The overall design focuses on robustness, usability, and scalability of query handling.
-

3. Pre-processing (Layer 2)

- User queries undergo preprocessing to remove noise and irrelevant characters before further processing.
- Text normalization ensures that queries are converted into a consistent format for better analysis.

- Tokenization techniques are applied to break the query into smaller meaningful components.
 - Stop-word removal may be applied to improve the relevance of processed queries.
 - Queries are prepared in a format suitable for embedding generation and retrieval operations.
 - Encoding inconsistencies and formatting errors are resolved during preprocessing.
 - NLP techniques are applied to improve semantic understanding of queries.
 - Preprocessing ensures that input data is clean and optimized for downstream tasks.
 - The stage plays a crucial role in improving system accuracy and performance.
 - It acts as a foundation for effective retrieval and model inference.
-

4. Retrieval (Layer 3)

- The retrieval layer is responsible for fetching relevant legal documents based on user queries.
- Dense retrieval is implemented using FAISS, which performs semantic similarity search using vector embeddings.
- Sparse retrieval is implemented using BM25, focusing on keyword-based matching.
- A hybrid retrieval approach combines both methods to improve accuracy and recall.

- Transformer-based models are used to generate embeddings for semantic understanding.
 - Similarity scores are computed between queries and documents to identify relevant results.
 - Top-k relevant documents are selected for further processing.
 - Separate indices are maintained for different document categories.
 - The retrieval system is optimized for fast response time and scalability.
 - This layer forms the backbone of the system's information access capability.
-

5. Reranking and AI Model Pipeline (Layer 4)

- Retrieved documents are passed through a cross-encoder reranking model for improved relevance.
- The reranking process evaluates each document in relation to the query more accurately.
- Cross-encoder models consider both query and document simultaneously for ranking.
- Only top-ranked documents are selected for final answer generation.
- A local GGUF-based large language model is used for generating responses.
- The model operates offline, ensuring data privacy and independence from external APIs.
- The entire pipeline is executed through Python scripts.
- Integration with Node.js backend is achieved using the `child_process` module.

- Model inference is optimized for efficiency and performance.
 - This layer is responsible for the intelligence and accuracy of the system.
-

6. Output Generation (Layer 5)

- The final response is generated by the language model using retrieved context.
 - Backend processes and formats the output for better readability.
 - Responses are structured to ensure clarity and relevance.
 - Irrelevant or redundant information is filtered out.
 - The system ensures consistent response formatting.
 - Error handling is implemented for fallback responses.
 - Outputs are designed to be user-friendly and understandable.
 - The system provides context-aware legal answers.
 - Output generation ensures high-quality responses.
 - This layer plays a key role in user satisfaction.
-

7. Backend Integration (Layer 6)

- The backend is implemented using Node.js and Express framework.
- It acts as the central control unit of the system.
- Python scripts are executed using the `child_process` module.
- Backend manages API requests and responses efficiently.

- Data flow between different layers is handled seamlessly.
 - Asynchronous processing ensures non-blocking operations.
 - Logging and error handling mechanisms are implemented.
 - Backend ensures smooth communication with AI components.
 - The architecture is designed for scalability and extensibility.
 - This layer forms the core infrastructure of the system.
-

8. Presentation Layer (Layer 7)

- The presentation layer is responsible for displaying results to users.
 - Future frontend implementation will provide a user-friendly interface.
 - Results will be displayed in structured and readable formats.
 - UI components will enhance user interaction.
 - Visualization features may be added for better understanding.
 - Frontend will communicate with backend via APIs.
 - Data will be presented in an intuitive manner.
 - UI/UX principles will be followed for better usability.
 - Interactive features may be integrated in future versions.
 - This layer enhances overall user experience.
-

9. Feedback and Improvement Loop

- A feedback mechanism can be implemented to improve system performance.
- User corrections can be stored for analysis and retraining.
- Collected data can be used to enhance model accuracy.
- Continuous learning techniques can be applied.
- Model versioning can be implemented for better tracking.
- A/B testing can be used to evaluate improvements.
- Feedback helps in refining system responses.
- The system can evolve based on user interaction.
- Performance optimization can be achieved over time.
- This layer ensures long-term improvement and adaptability.

3. Conclusion and Future Work

3.1 Conclusion

By the mid-semester checkpoint, the LegalSahyak project has successfully established a strong architectural foundation and a fully functional backend system. The system is designed around a multi-layered pipeline that integrates hybrid retrieval techniques with local large language model inference, ensuring efficient and privacy-preserving legal query processing.

The backend, developed using Node.js and Express, is capable of handling end-to-end query processing, starting from user input to final response generation. The integration of the Python-based machine learning pipeline using the `child_process` module demonstrates seamless communication between backend services and AI components. Additionally, the implementation of hybrid retrieval using FAISS and BM25, along with reranking techniques, ensures that

the system can retrieve and process relevant legal information with high accuracy.

At its current stage, the system is capable of executing the complete pipeline, including query processing, document retrieval, reranking, and response generation using a local GGUF-based model. The APIs have been tested and validated using Postman, ensuring reliability and correctness of the backend system. Overall, the project has successfully achieved its mid-semester objectives by building a scalable and modular backend integrated with AI capabilities, providing a solid foundation for further development into a complete end-to-end application.

3.2 Future Work

Although the LegalSahyak system has achieved significant progress in backend development and AI integration, several important components and enhancements remain to be implemented to transform it into a complete production-ready system.

Frontend Development

- Development of a user-friendly frontend interface is one of the major pending tasks.
 - The frontend will allow users to input legal queries and view generated responses in an intuitive manner.
 - Integration of frontend with backend APIs will enable real-time query processing and response display.
 - UI/UX design improvements will ensure accessibility and ease of use for non-technical users.
-

Full System Integration

- Integration of frontend, backend, and AI pipeline into a single unified system is required.
 - Ensuring smooth communication between all components will be critical for system stability.
 - End-to-end testing of the complete system will be performed after integration.
-

Dataset Expansion and Improvement

- Expanding the legal dataset to include more statutes, acts, and case laws.
 - Improving data quality and coverage for better retrieval accuracy.
 - Automating data scraping and ingestion pipeline for continuous dataset updates.
-

Model Optimization and Performance Enhancement

- Optimization of the GGUF-based LLM for faster inference and reduced latency.
 - Improving retrieval efficiency and reranking performance.
 - Exploring lightweight models for better performance on low-resource systems.
-

Advanced Features

- Implementation of multi-query support and conversational interaction.
 - Adding support for multilingual legal queries.
 - Enhancing response explainability by showing source documents or references.
-

System Deployment

- Deployment of the system as a desktop or web-based application.
 - Ensuring compatibility across different platforms (Windows, Android, etc.).
 - Packaging the system for offline usage without complex setup.
-

Monitoring and Logging

- Implementing logging systems to track queries and responses.
 - Monitoring system performance and identifying bottlenecks.
 - Using logs for debugging and continuous improvement.
-

User Feedback and Learning Loop

- Implementing a feedback mechanism to collect user responses.
- Using feedback data to improve model performance over time.

- Introducing retraining pipelines for continuous system improvement.
-

Security and Reliability

- Ensuring secure handling of user queries and data.
 - Implementing error handling and fallback mechanisms.
 - Improving system robustness for real-world usage scenarios.
-

Scalability and Future Extensions

- Designing the system to support larger datasets and higher query loads.
 - Adding support for integration with other legal tools or platforms.
 - Extending the system to support document-based queries and uploads.
-

References

- [1] LegalSahyak Project Documentation and README
 - [2] FAISS (Facebook AI Similarity Search) Documentation
 - [3] BM25 Retrieval Algorithm
 - [4] LangChain Framework
 - [5] Llama.cpp (GGUF Model Inference)
 - [6] India Code Legal Dataset
-

References

- [1] LegalSahyak Project Documentation and README File, 2026.
- [2] FAISS (Facebook AI Similarity Search) – Official Documentation, Meta AI.
- [3] BM25 Retrieval Algorithm – Information Retrieval Techniques.
- [4] LangChain Framework – Official Documentation and Usage Guides.

- [5] Llama.cpp – GGUF Model Inference Framework Documentation.
- [6] India Code – Government of India Legal Dataset Repository.
- [7] Sentence Transformers (BAAI/bge models) – Embedding Model Documentation.
- [8] Cross-Encoder Reranking Models – HuggingFace Transformers Documentation.
- [9] Node.js and Express.js – Official Backend Development Documentation.
- [10] Python Libraries (pandas, scikit-learn) – Official Documentation.